Hibernate III

-By Miss. Geetanjali Gajbhar

HQL:

- Hibernate Query language.

Topic	SQL	HQL
Get All Data	SELECT * FROM table_name;	FROM class_name;
Single Single Data	SELECT * FROM table_name WHERE col_name=value;	FROM class_name WHERE variable_name=value;
Update Data	UPDATE table_name SET col_name=value WHERE col_name=value;	UPDATE class_name SET variable_name=value WHERE variable_name=value;
Delete Data	DELETE FROM table_name WHERE col_name=value;	DELETE FROM class_name WHERE variable_name=value;

HQL vs SQL:

- Internally uses **SQL**.
- In SQL we use Table_name, Column_name in Query, But in HQL we Use **Class_name, property_name**
- HQL is **DB independent** language but SQL is DB Dependent language.
- HQL works with **persistent Object**, SQL works with Table and column.
- HQL Supports **OOPs**.
- HQL works with **persistent object** (Which are associate with DB and session both).

Here we will use Java Based Approach For Configuration

Chapter 1 CRUD using HQL

Get All Data

Use **getResultList() method.**

Create a Student table and insert data into it at DB end. And now we will try to retrieve the Student data using HQL query.

HibernateUtil class:

- Here get SessionFactory is a static method which returns **Sessionfactory Object.**
- Hence Outside the class We can call this method using the class name

`SessionFactory sf=HibernateUtil.getSessionFactory();`

- **HibernateUtil class:**

```
package com.model;
import java.util.HashMap;
import java.util.Map;
import org.hibernate.SessionFactory;
import org.hibernate.boot.Metadata;
import org.hibernate.boot.MetadataSources;
import org.hibernate.boot.registry.StandardServiceRegistry;
import org.hibernate.boot.registry.
        StandardServiceRegistryBuilder;
import org.hibernate.cfg.Environment;

public class HibernateUtil {
    private static SessionFactory sf = null;
    private static StandardServiceRegistry registry;

    public static SessionFactory getSessionFactory() {

        // connction properties
        try {
            if (sf == null) {

                Map<String, String> map = new HashMap<>();
    map.put(Environment.DRIVER, "com.mysql.jdbc.Driver");
        map.put(Environment.URL,
"jdbc:mysql://localhost:3306/test");
                map.put(Environment.USER, "root");
                map.put(Environment.PASS, "root");

                // hibernate properties

                map.put(Environment.DIALECT,
"org.hibernate.dialect.MySQLDialect");
```

```
                map.put(Environment.HBM2DDL_AUTO, "update");
                map.put(Environment.SHOW_SQL, "true");
                map.put(Environment.FORMAT_SQL, "true");

                // create object of StandardServiceRegistryBuilder

                registry = new StandardServiceRegistryBuilder()
.applySettings(map).build();

                // create object of Metadatasources

        MetadataSources mds = new MetadataSources(registry);

//-----------------------------------------
        mds.addAnnotatedClass(Student.class);
//-----------------------------------------

                // create object of Metadata
                Metadata md = mds.getMetadataBuilder().build();
                sf = md.getSessionFactoryBuilder().build();
            }
        }

        catch (Exception e) {
            e.printStackTrace();
        }
        return sf;

    }

}
```

- Test Class

```
package com.model;

import java.util.ArrayList;
import java.util.List;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;
import org.hibernate.query.Query;

public class Test1 {
    public static void main(String[] args) {

SessionFactory sf = HibernateUtil.getSessionFactory();
```

```
Session session = sf.openSession();

        // ------------------------------------------------------

// create student table and insert data into it at DB end.
// try to get the student data Using HQL query
        String hql = "from Student";
        Query query = session.createQuery(hql);
        List<Student> list = query.getResultList();
        for (Student s : list) {
            System.out.println(s.getId());
            System.out.println(s.getName());
            System.out.println(s.getCity());
        }

        System.out.println("data fetched!!");

        // close the session
        session.close();

    }

}
```

- You will get all the Student data on the console.

Get single Data

Use **getSingleResult**() method

- If we want to retrieve single student data
- Test.class

```
        // try to get the single student data Using HQL query

        String hql = "from Student where id=102";
        Query query = session.createQuery(hql);
    Student s = (Student) query.getSingleResult();

            System.out.println(s.getId());
            System.out.println(s.getName());
            System.out.println(s.getCity());


        System.out.println("data fetched!!");
```

Extra:

Query(I) has some Methods

```
query.getFirstREsult();-> gives frist result
query.executeUpdate(); -> execute the query
query.getResultList(); -> to retrieve multiple data
query.list();          -> to retrieve multiple data
query.getSingleResult();-> to retrieve single data
```

Update Data

- Here we need Transaction

Test.class

```
package com.model;

import java.util.ArrayList;
import java.util.List;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;
import org.hibernate.query.Query;

public class Test1 {
    public static void main(String[] args) {

SessionFactory sf =
HibernateUtil.getSessionFactory();

Session session = sf.openSession();
Transaction tx = session.beginTransaction();
        // ------------------------------------------------------


String hql =
 "update Student set name='Mohit' where id=103";

Query query = session.createQuery(hql);
        query.executeUpdate();
        tx.commit();
        System.out.println("data Updated!!");
```

```
        // close the session
        session.close();

    }

}
```

Delete Data

- Here we need Transaction
- Test.class

```
package com.model;

import java.util.ArrayList;
import java.util.List;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;
import org.hibernate.query.Query;

public class Test1 {
    public static void main(String[] args) {

SessionFactory sf =
 HibernateUtil.getSessionFactory();
Session session = sf.openSession();
Transaction tx = session.beginTransaction();
        // ------------------------------------------------------


String hql = "delete from Student where id=103";
        Query query = session.createQuery(hql);
        query.executeUpdate();
        tx.commit();
        System.out.println("data Deleted!!");

        // close the session
        session.close();

    }
```

```
}
```

Insert Data

- HQL is for persistent Object. We cannot add new Object directly but we can add the persistent object from another table.

```
"INSERT INTO table_1(id,name,city)"+
"SELECT id,name,city FROM table_2"
```

Chapter 2

Pagination in Hibernate

Pagination is **a simple t feature to limit the size of your result set.**

You can configure it with JPA and Hibernate by calling the **setFirstResult** and **setMaxResults**.

Test Class

```
package com.model;

import java.util.ArrayList;
import java.util.List;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;
import org.hibernate.query.Query;

public class Test1 {

    public static void main(String[] args) {
        SessionFactory sf =
 HibernateUtil.getSessionFactory();

        Session s = sf.openSession();
        Transaction tx = s.beginTransaction();
        // ------------------------------------------------
        String hql = "from Student";
        Query q = s.createQuery(hql);

        // pagination
        q.setFirstResult(0);// index of first row
        q.setMaxResults(3);// 0,1,2,
```

```
        List<Student> list = q.list();
        for (Student a : list) {
            System.out.println(a.getId() + "--" +
 a.getName() + "--" + a.getCity());
        }

        System.out.println("pagination Achieved!!");
        // ------------------------------------------
        s.close();

    }

}
```

101--Aman--Surat
102--Ajeet--Mumbai
103--Jamnagar--Mohit

Chapter 3

Executing Native SQL query using Hibernate

- `Session.creayeQuery(HQLquery)` This method is for HQL query. And return type of this method is **Query**
- `session.createSQLQuery(SQL_query)` This method us for SQL query, And return type of this method is **NativeQuery**

Test class

```
package com.model;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;
import org.hibernate.query.NativeQuery;
import org.hibernate.query.Query;

public class Test1 {

    public static void main(String[] args) {
        SessionFactory sf = HibernateUtil.getSessionFactory();
        Session s = sf.openSession();

        // ------------------------------------------
        String sql = "select * from Student where id=102";
```

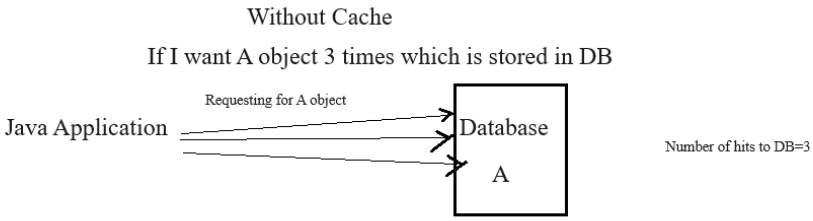
```
        NativeQuery nq = s.createSQLQuery(sql);

        List<Object[]> list = nq.list();
        for (Object[] st : list) {
            System.out.println(Arrays.deepToString(st));

        }

        // ------------------------------------------
        s.close();

    }

}
```

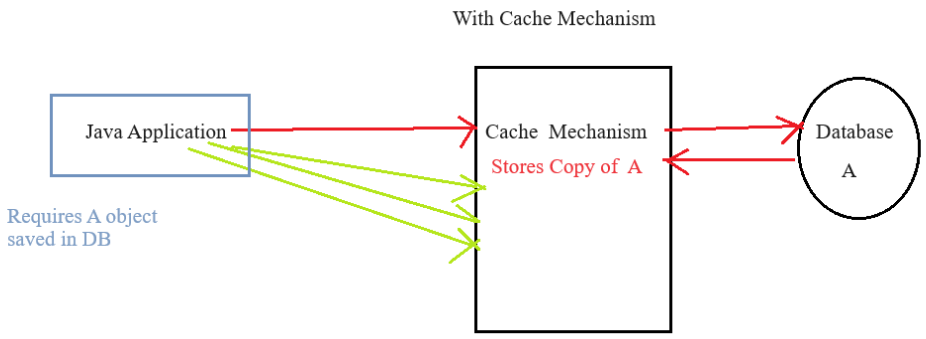
[102, Mumbai, Ajeet]

Chapter 4

Hibernate Caching

- Caching mechanism is used to **Enhance the performance** of an application. **by reducing** the number of hits to database.





We can clearly see Number of hits to DB are only 1

Hibernate provides two levels of Caching

1. **First level Cache**
2. **Second Level Cache**

First Level Cache:

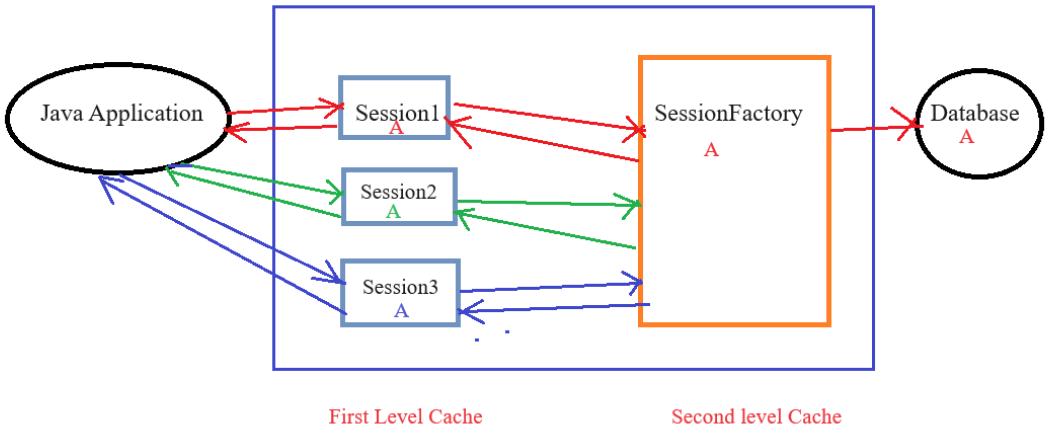
- It is associated with **Session Object.**
- First level cache is bycdefault enable.

Second level Cache:

- It is associated with **SessionFactory Object.**
- Second level cache is by cdefault not available. We have to enable it manually.
- For that add **ehCache** Jar. Add some annotations.

```
@Cacheable
@Cache(usage=cacheConcurrencyStrategy.Read_only)
@Entity
public class Student
{

}
```

- or an entire project we have **only one** SessionFactory object, and **multiple** session objects.
- SessionFactory Object is **Thread safe.**



Chapter 5

Criteria Builder API

- Developer friendly way to communicate to DB.
- No need to Have SQL/HQL knowledge. No need to write Queries.
- This API provides some Methods to communicate with Database.

Get All Data using Criteria Builder

Test.java

```
package com.model;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

import org.hibernate.Criteria;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;
import org.hibernate.query.NativeQuery;
import org.hibernate.query.Query;

public class Test1 {

    public static void main(String[] args) {
        SessionFactory sf = HibernateUtil.getSessionFactory();
        Session session = sf.openSession();

        // ------------------------------------------------
```

```
        // provide entity class
        Criteria c = session.createCriteria(Student.class);

        List<Student> l = c.list();

        l.forEach(st -> {
            System.out.println(st.getId());
            System.out.println(st.getName());
            System.out.println(st.getCity());
        });
        // ------------------------------------------
        session.close();

    }

}
```

101--Aman--Surat
102--Ajeet--Mumbai
103--Jamnagar--Mohit
104--Ram--Dhule
105--OM--Latur

We can apply restrictions like get the data whose city=”mumbai” etc.

Test.class

```
package com.model;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

import org.hibernate.Criteria;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;
import org.hibernate.criterion.Restrictions;
import org.hibernate.query.NativeQuery;
import org.hibernate.query.Query;

public class Test1 {

    public static void main(String[] args) {
SessionFactory sf = HibernateUtil.getSessionFactory();
Session session = sf.openSession();

        // ------------------------------------------
```

```
        // provide entity class
Criteria c = session.createCriteria(Student.class);
        //Restriction
        c.add(Restrictions.eq("city", "Mumbai"));
        List<Student> l = c.list();
        l.forEach(st -> {
            System.out.println(st.getId());
            System.out.println(st.getName());
            System.out.println(st.getCity());
        });
        // ------------------------------------------
        session.close();

    }

}
```

102--Ajeet--Mumbai

Some restrictions in Criteria Builder:

- .eq()
- .like()
- .isNull()
- .notNull()
- .between()

Chapter 6

Inheritance in Hibernate(Is-A Relation)

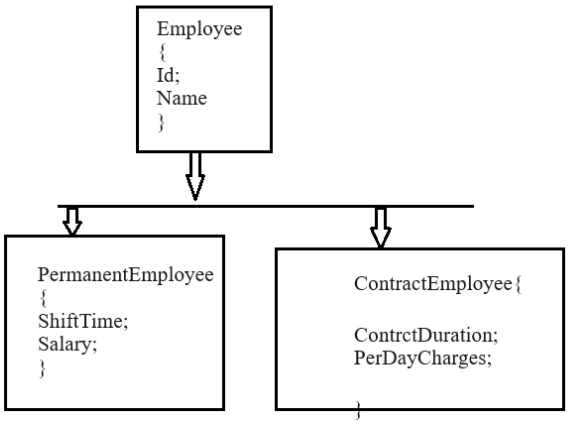
- In hibernate if we have P-C classes then if we save child's object automatically parent class's object will be saved.

Hibernate Support 4 types of Inheritance strategies

1. Mapped Super class→ 2 Tables get created
2. Single Table→ 1 table get create and one extra column of type.
3. Joined Table→ 3 tables get created.
4. Table Per class→In case of table per concrete class, tables are created as per class. But duplicate column is added in subclass tables.

Example:

- @Inheritance(strategy=InheritanceType.SINGLE_TABLE)



- Employee Class**(Parent class)**

```
@Entity
@Inheritance(strategy=InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name="type",discriminatorType=DiscriminatorType.STRING)
@DiscriminatorValue(value="employee")

public class Employee {
@Id
@GeneratedValue(strategy=GenerationType.AUTO)
private int id;
private String Name;

//getter setter construtor
}
```

RegularEmployee class

```
@Entity
@DiscriminatorValue("regularemployee")
public class Regular_Employee extends Employee{
ShiftTime;
salary;
}
```

ContrctEmployee class

```
@Entity
@DiscriminatorValue("contractemployee")
public class Contract_Employee extends Employee{

contractDuration;
```

```
perDayCharges;
}
```

- Add 3 Mapping resources in Configuration file.

Test.class

```
public class Test{
public static void main(String[] args){

//carete SF object
//get Session object

//Create EMployee Object set id and name
//create RegularEmployee class object set values
id
name
shiftTime
salary

//create ContractEmployee class object set values
id
name
ContractDuration
PerDay Charge

session.save(EmployeeObject);
session.save(RegularEmployeeObject);
session.save(ContractEmployeeObject);
tx.commit();
Session.close();


}
}
```

Output on DB end

Type	id	name	shiftTime	salary	contractDuration	perDaycharge
Employee	101	Gaurav	-	-	-	-
RegularEmployee	102	Pritam	5-9	50000	-	-
ContractEmployee	103	Amar	-	-	6months	700/-

Extra:

What is Dirty Checking in Hibernate?

If we modify the object in trasaction boundry and commit the changes then its state automatically changes without calling session.update(). Internally Update query get executed. This without permission updation is called

as Dirty Checking.

Chapter 7

Stored Procedure in Hibernat

Example: Create a Stored Procedure at DB end with name selectAllData

Test.class

```
package cpm.client;

import java.util.List;

import javax.persistence.ParameterMode;
import javax.persistence.StoredProcedureQuery;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;

import com.config.HibernateUtil;
import com.model.Student;

public class SelectAllData {
    public static void main(String[] args) {

        SessionFactory sf=HibernateUtil.getSessionFactory();
        Session session =sf.openSession();
        Transaction tx=session.beginTransaction();
//------------------------------------------------------------

        StoredProcedureQuery query=
session.createStoredProcedureCall
("selectAllData",Student.class);

        List<Student> list=query.getResultList();
        for(Student student:list) {
            System.out.println(student.getId());
            System.out.println(student.getName());
        }

    }

}
```

Example: Create a Stored Procedure at DB end with name inserData

Test.class

```
package cpm.client;

import javax.persistence.ParameterMode;
import javax.persistence.StoredProcedureQuery;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;

import com.config.HibernateUtil;

public class Test {
    public static void main(String[] args) {
        SessionFactory sf=HibernateUtil.getSessionFactory();
        Session session =sf.openSession();
        Transaction tx=session.beginTransaction();

        StoredProcedureQuery query=
session.createStoredProcedureCall("insertData");

        query.registerStoredProcedureParameter
("sid", Integer.class, ParameterMode.IN);

        query.registerStoredProcedureParameter
("nm", String.class, ParameterMode.IN);

        query.setParameter("sid", 3);
        query.setParameter("nm", "Sam");
        query.execute();
        tx.commit();


    }

}
```

Example: Create a Stored Procedure at DB end with name updateData

```
package cpm.client;

import javax.persistence.ParameterMode;
import javax.persistence.StoredProcedureQuery;

import org.hibernate.Session;
```

```
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;

import com.config.HibernateUtil;

public class UpdateData {
    public static void main(String[] args) {
        SessionFactory sf=HibernateUtil.getSessionFactory();
        Session session =sf.openSession();
        Transaction tx=session.beginTransaction();

        StoredProcedureQuery query=
session.createStoredProcedureCall("updateData");

        query.registerStoredProcedureParameter
("nm", String.class, ParameterMode.IN);

        query.registerStoredProcedureParameter
("id", Integer.class, ParameterMode.IN);

        query.setParameter("nm", "omkar");
        query.setParameter("id", 3);

        query.executeUpdate();
        tx.commit();
    }

}
```

Task:

1. CRUD using Hibernate Methods
2. CRUD using Hibernate NamedQueries
3. CRUD using HQL
4. CRUD using Stored Procedure.